CSE 242 — Assignment 3: SVMs, K-means, and K-NN

Ioannis Barous · Student ID 2252832 · CSE 242 · Assignment 3

Question 1: Hard-margin linear SVM

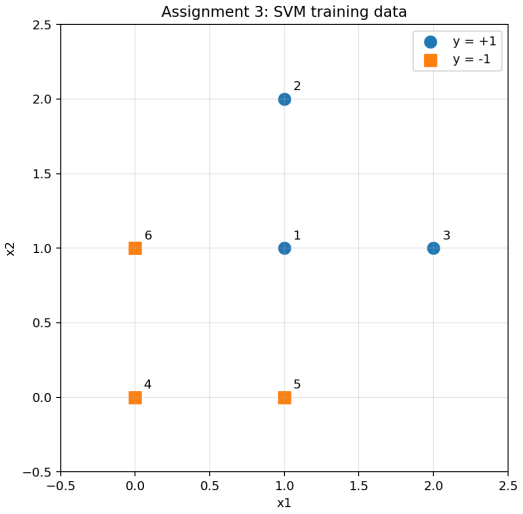


Figure 1. Training data (svm_data.png).

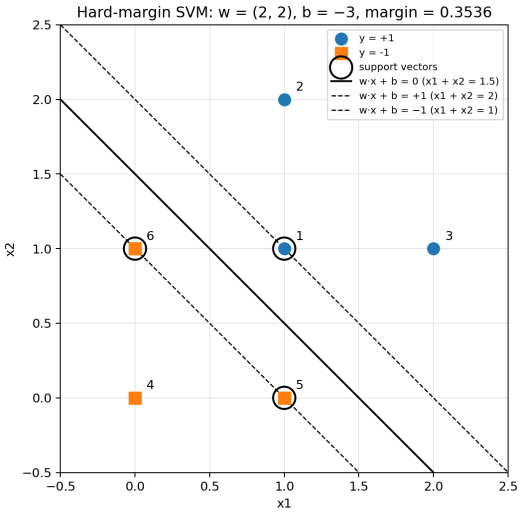


Figure 2. Max-margin boundary, margin lines, and support vectors (circled).

(a) Support vectors

The support vectors are **points 1 (1, 1), 5 (1, 0), and 6 (0, 1)**. The gap between the classes runs diagonally. Point 1 is the positive point closest to the negative class, and points 5 and 6 are the negative points closest to the positive class; they lie on the line $x_1 + x_2 = 1$, while point 1 lies on the parallel line $x_1 + x_2 = 2$. The widest strip separating the classes lies between these two lines, so these three points determine the margin. Points 2 and 3 (on $x_1 + x_2 = 3$) and point 4 (the origin) are farther away; moving them slightly would not change the boundary.

(b) Decision boundary

For a hard-margin SVM every point satisfies $y_i(w \cdot x_i + b) \geq 1$, with equality for the support vectors. One equation per support vector:

Point 1, x = (1, 1), y = +1: $(+1)(w_1 + w_2 + b) = 1$ so $w_1 + w_2 + b = 1$ (i)

Point 5, x = (1, 0), y = −1: $(-1)(w_1 + b) = 1$ so $w_1 + b = -1$ (ii)

Point 6, x = (0, 1), y = −1: $(-1)(w_2 + b) = 1$ so $w_2 + b = -1$ (iii)

Solving: (i) − (ii) gives $w_2 = 2$; (i) − (iii) gives $w_1 = 2$; then (ii) gives $b = -1 - 2 = -3$. So **$w = (2, 2)$, $b = -3$**, and the decision boundary is **$2x_1 + 2x_2 - 3 = 0$** (equivalently $x_1 + x_2 = 1.5$). The supporting lines are $w \cdot x + b = +1$ ($x_1 + x_2 = 2$, through point 1) and $w \cdot x + b = -1$ ($x_1 + x_2 = 1$, through points 5 and 6).

Optimality check. With $w = \Sigma\, \alpha_i y_i x_i$ and $\Sigma\, \alpha_i y_i = 0$: $\alpha_1 - \alpha_5 - \alpha_6 = 0$, $\alpha_1 - \alpha_5 = 2$, $\alpha_1 - \alpha_6 = 2$, giving $\alpha_5 = \alpha_6 = 2$ and $\alpha_1 = 4$. All are positive, so the KKT conditions hold and this is the optimal hard-margin solution. Every point satisfies the constraints, with equality only at the support vectors:

Point	x	y	w·x + b	y(w·x + b)
1	(1, 1)	+1	1	1 (SV)
2	(1, 2)	+1	3	3
3	(2, 1)	+1	3	3
4	(0, 0)	−1	−3	3
5	(1, 0)	−1	−1	1 (SV)
6	(0, 1)	−1	−1	1 (SV)

Geometric margin

$||w|| = \sqrt{(2^2 + 2^2)} = \sqrt{8} = 2\sqrt{2} \approx 2.8284$.

> **Geometric margin** (distance from the decision boundary to the closest training points): $\gamma = 1/||w|| = 1/(2\sqrt{2}) = \sqrt{2}/4 \approx$ **0.3536**. Check with point 1: $|2 + 2 - 3| / (2\sqrt{2}) \approx 0.3536$.
>
> **Full margin width** (distance between the two supporting lines, reported separately): $2/||w|| = 1/\sqrt{2} \approx$ **0.7071**.

Running `python assignment3_template.py --task svm` prints support vectors [1, 5, 6], weights (2.0, 2.0), bias −3.0, and geometric margin 0.35355339059327373. The function builds the three support-vector equations and solves them; no library SVM is used. As a separate check only, sklearn SVC(kernel="linear", C=1e6) gave the same w, b, and support vectors.

Question 2: K-means

(a) Implementation

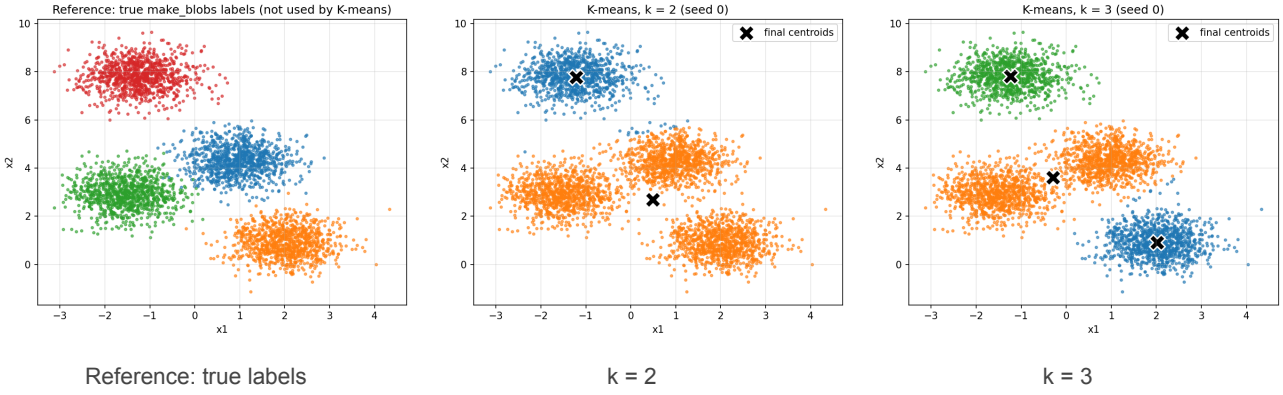
update_assignments() computes the squared Euclidean distance from every point to every centroid with NumPy broadcasting and assigns each point to the nearest centroid. update_centroids() sets each nonempty cluster's centroid to the mean of its points. No library K-means is used.

> **Tie rule:** a point equally distant from two or more centroids is assigned to the centroid with the lowest index (np.argmin returns the first minimum). This also covers duplicate initial centroids: only the lower-index copy receives points.
>
> **Empty-cluster rule:** a cluster with no assigned points keeps its previous centroid. The input centroid array is copied and never modified in place.

(b) Results (seed 0, 100 iterations)

Data: make_blobs(n_samples=4000, centers=4, cluster_std=0.60, random_state=0). One generator np.random.default_rng(0) supplies the initial centroids for every k via get_initial_clusters(). The true labels are used only in the reference plot. Every run converged (one more update changes nothing), and no run started with duplicate centroids or ended with an empty cluster.

k	Inertia (sum of squared distances)	Cluster sizes
2	15222.21	1017, 2983
3	6826.13	1008, 1992, 1000
4	2794.24	1000, 998, 1002, 1000
6	2396.79	1000, 299, 997, 407, 319, 978
10	1914.79	365, 350, 253, 261, 988, 997, 123, 213, 285, 165



Reference: true labels | k = 2 | k = 3

k = 4 | k = 6 | k = 10

Figure 3. K-means assignments (colors) and final centroids (black X) for each k.

Effect of k

As k increases, inertia drops (15222 → 6826 → 2794 → 2397 → 1915), but the large drops stop at k = 4, the number of generating groups. With **k < 4**, K-means must merge real blobs: at k = 2 the isolated top blob gets its own cluster and the other three are merged into one cluster of 2983 points with a centroid in the empty space between them; at k = 3 the bottom-right blob separates, but the two middle blobs stay merged. At **k = 4** the clusters match the true blobs almost exactly: sizes 1000/998/1002/1000, only 9 of 4000 points grouped with a different blob than their true one, and centroids equal to the true blob means to two decimals. With **k > 4**, K-means splits real blobs, and the splits depend on the random initialization: at k = 6 the left-middle blob is cut into three pieces while the other three stay whole; at k = 10 the bottom-right blob is cut into five pieces and the top blob into three, while the two middle blobs stay whole. Inertia keeps decreasing, but the extra clusters only fragment real groups. No run got stuck with two centroids in one blob while two other blobs were merged.

Question 3: Noisy-label SVM and K-NN repair

Setup

- Training and test sets are generated once and kept fixed, each with 1000 positive examples from N((1, 1), I) and 1000 negative examples from N((−1, −1), 3I). Test labels stay clean.
- Label noise (training only): each original +1 label is flipped with probability 0.35 and each original −1 label with probability 0.20, using one uniform draw per example so no label is flipped twice.
- Classifier: RBF SVM with C = 1 and gamma = 0.01, trained on noisy labels (baseline) and on K-NN-repaired labels.
- K-NN repair: for each training point, its 5 nearest *other* training points (Euclidean, sklearn NearestNeighbors) vote with their noisy labels and the majority replaces the label. All repairs use the same original noisy array. Distance ties are broken by lower row index. The repair uses only training features and noisy labels, never clean or test labels.
- 20 trials of corruption + training, seed 0 (np.random.default_rng(0)); each trial uses the same noisy labels for both methods. Command: `python noisy_classification_template.py --repeats 20 --seed 0`.

Results

Trial	Baseline	K-NN	Diff.
1	0.8345	0.8615	+0.0270
2	0.8330	0.8720	+0.0390
3	0.8440	0.8675	+0.0235
4	0.8445	0.8700	+0.0255
5	0.8395	0.8725	+0.0330
6	0.8410	0.8715	+0.0305
7	0.8405	0.8690	+0.0285
8	0.7930	0.8480	+0.0550
9	0.8240	0.8615	+0.0375
10	0.8425	0.8715	+0.0290

Trial	Baseline	K-NN	Diff.
11	0.8220	0.8600	+0.0380
12	0.8190	0.8645	+0.0455
13	0.8470	0.8645	+0.0175
14	0.8415	0.8715	+0.0300
15	0.8160	0.8715	+0.0555
16	0.8285	0.8485	+0.0200
17	0.8235	0.8695	+0.0460
18	0.8430	0.8700	+0.0270
19	0.7840	0.8335	+0.0495
20	0.7980	0.8690	+0.0710

	Mean test accuracy (20 trials)
Baseline (noisy labels)	0.82795
K-NN repair	0.864375
Difference (K-NN − baseline)	+0.036425

Exact means of the 20 CSV values. The script prints them rounded to four decimals: 0.8280, 0.8644, and +0.0364.

K-NN repair improved test accuracy in 20 of 20 trials. For reference, the same SVM trained on the clean training labels reaches 0.8670 on this test set.

Discussion

K-NN repair helped here, but not by making the training labels more correct overall. Measured against the clean training labels (used only for this analysis, never by the repair), the average label error was 0.270 before repair and 0.273 after: each trial fixed about 382 wrong labels and broke about 387 correct ones. The effect differs by region:

- **Helps where classes are well separated.** A random flip is usually outvoted by its correctly labeled neighbors (label error there fell from 0.269 to 0.228). The baseline SVM instead sees scattered flipped points deep inside the other class, which pull its boundary off.
- **Hurts where classes overlap.** There the neighbors' votes are close to a coin flip, so the majority is often wrong (label error rose from 0.279 to 0.509 where the two class densities are similar). The negative class (covariance 3I) is much more spread out and extends into the positive region, so many genuine negatives there are relabeled +1.
- **Asymmetric noise biases the vote.** With 35% of positives flipped but only 20% of negatives, the noisy labels are only 43.1% positive instead of 50%, so near the boundary the majority tends toward −1; after repair 41.1% of labels are positive.
- **Why the SVM still improves.** Repair makes labels spatially consistent: each region tends to get the label of the class more likely there. Disagreement with the Bayes-optimal rule (computed from the known Gaussians) dropped from 0.331 to 0.233, and the SVM learns this smoother labeling better than randomly scattered flips. The gain is not guaranteed: with more class overlap or more lopsided noise, errors in the overlap region could outweigh the fixed flips.

Acknowledgment of outside help

I used Claude (Anthropic's AI assistant) to help with the code and the writeup for this assignment. The full chat transcript is attached.

Note: requirements.txt does not include scikit-learn, which the provided code uses, so I installed scikit-learn (and pandas) separately in the virtual environment.